

PLATFORM ARCHITECTURE · DELIVERY REPORT

Multi-Tenancy in the PGR Platform

How one platform now securely serves many institutions — the data model, the safeguards, and the proof.

95

tables with enforced
isolation

4

database environments

100%

DB-level enforcement

558

automated tests passing

Status: Delivered & verified

Scope: Backend data model, row-level security, tenant routing, environments

Date: 23 September 2026

Executive summary

The platform can now host **many institutions in a single system** while keeping each institution's data completely separate. Separation is enforced by the database itself — not by application code — so it holds even if a query is written incorrectly.

What changed

- Every institution's record is tagged with the institution it belongs to (a `tenant_id`).
- The database refuses to return or modify another institution's rows — automatically, on every request.
- Each institution is reached at its own web address (subdomain), and a login for one institution cannot be used on another's.

Why it matters

- **Trust & compliance:** a hard, provable data boundary between customers.
- **Efficiency:** one system to run and maintain, not one per customer.
- **Growth:** onboard a new institution as configuration, not a rebuild.

This report shows the data model that makes this possible and the live evidence that it works.

How tenants are kept separate in one database

All institutions share one database and even the same tables. What keeps them apart is a rule the database applies to every read and write: *return only the rows that belong to the institution making the request*. Rows for other institutions are physically present but invisible.

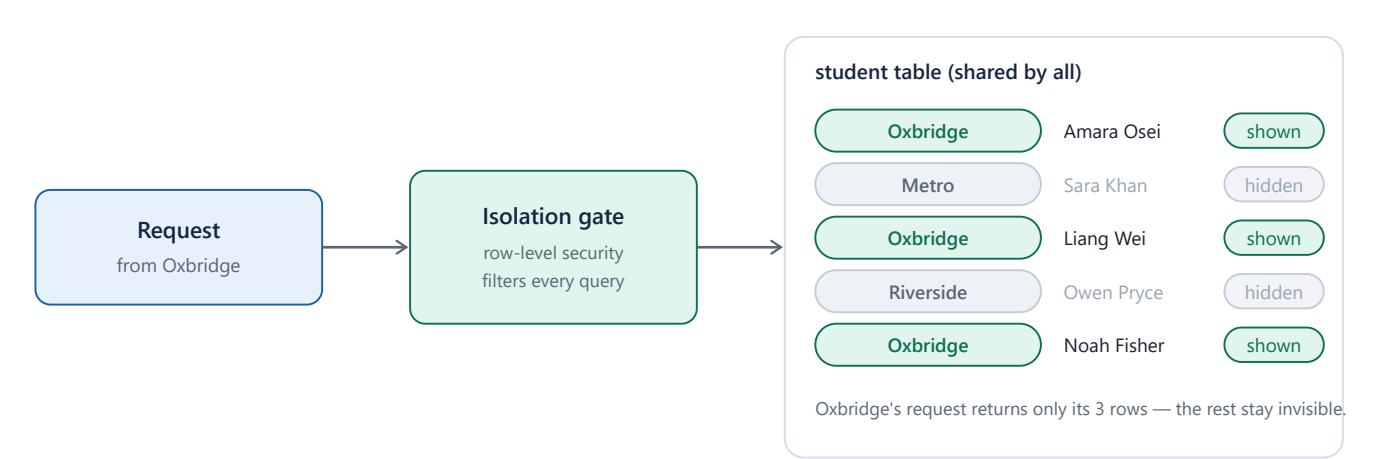


Figure 1 — One shared table; the database returns only the requesting institution's rows.

The data model

A single **tenant** table lists the institutions. Every institution-owned table then carries a `tenant_id` that points back to it, and each of those tables has a security policy attached. A small set of shared reference tables (roles, permissions, national reporting specs) are deliberately not tenant-specific.

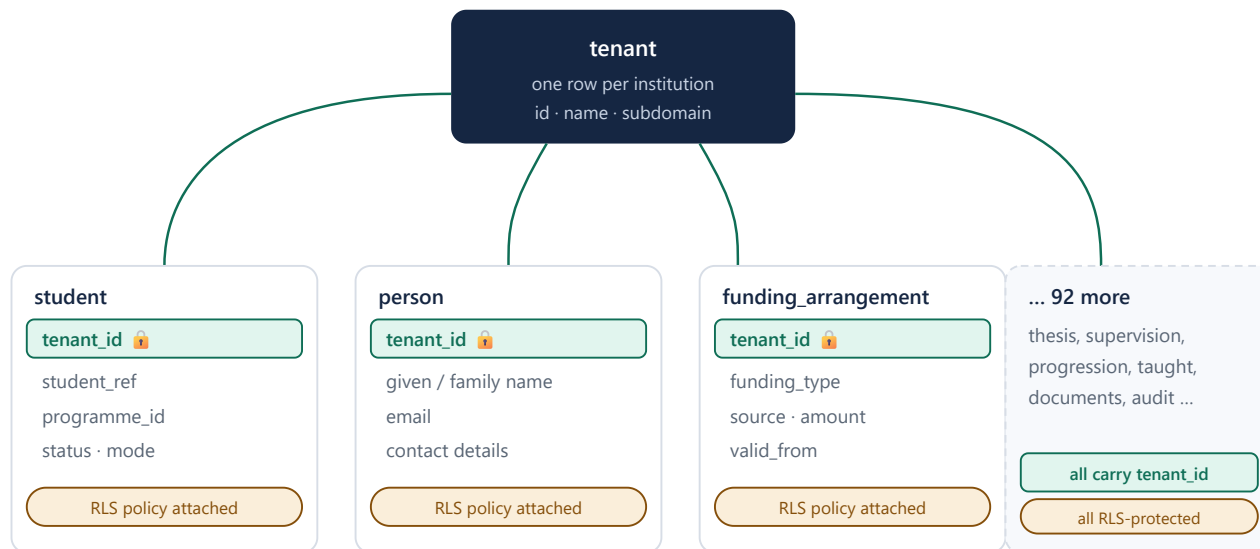


Figure 2 — One **tenant** table; every institution-owned table carries `tenant_id` and a row-level security policy (95 tables in total).

Tenant-owned (95 tables)

students, people, programmes, supervision, progression, funding, thesis, taught modules, documents, audit, analytics — every table that holds an institution's data.

Shared reference (not tenant-specific)

the tenant directory itself, roles & permissions, sign-in tokens, and national statutory reporting specifications.

How a request stays isolated, step by step

When someone uses the platform, the system establishes *which institution* the request is for, tells the database, and the database enforces the boundary for the rest of that request.

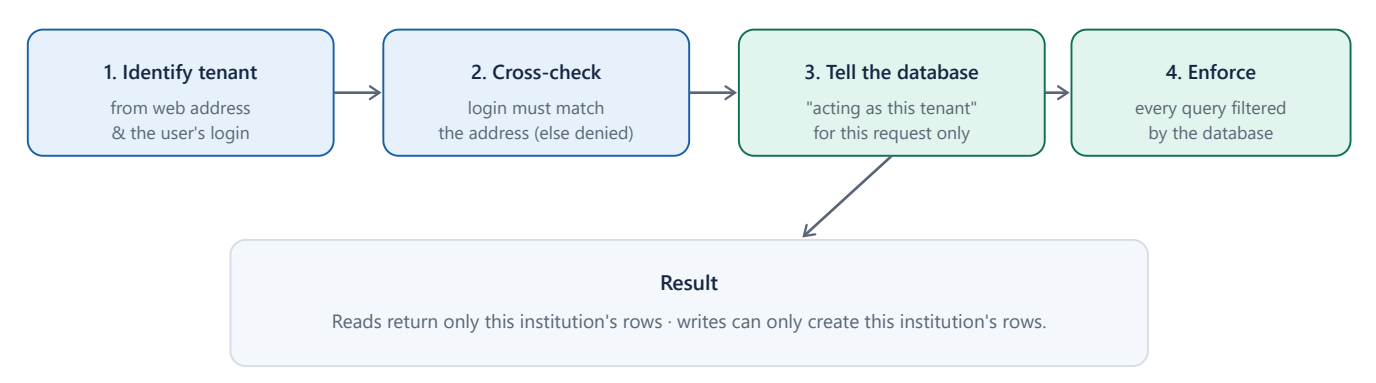


Figure 3 — The request pipeline: identify → cross-check → declare → the database enforces.

A defence-in-depth detail: the enforcement lives on the database tables, so it applies to every path — screens, reports, exports, and ad-hoc queries alike.

Before and after

Aspect	Before (skeleton)	After (delivered)
Institution tagging	only on the sign-in table	every one of 95 data tables
Isolation enforcement	none — effectively single-institution	enforced by the database on all tables
Web addressing	not tenant-aware	per-institution subdomain + cross-tenant block
Cross-institution leak risk	a missed filter would expose data	the database refuses it regardless
Onboarding a new institution	would need a rebuild	configuration — add a tenant

Four database environments

The same tenant-isolated design is deployed across four separate databases — the standard development, testing, production and backup stages. Each one is itself a shared, isolated multi-tenant database; the four are deployment stages, **not** one-per-tenant.

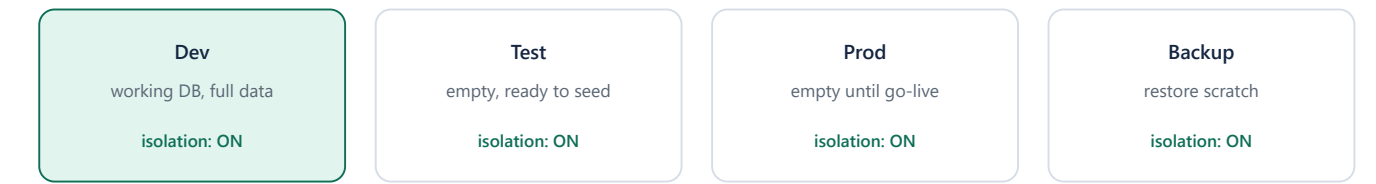


Figure 4 — Identical isolated schema across all four environments (107 tables, 261 relationships, 95 protected tables each).

Live evidence of isolation

To prove the boundary holds, four institutions were loaded into the **same** database and each was asked what it could see. Every institution saw only its own records; an unknown institution saw nothing.

Signed in as	Programmes visible	Students visible	Verdict
Default institution	5	671	own data only
Oxbridge University	1	3	own data only
Metro College	1	3	own data only
Riverside Institute	1	3	own data only
Unknown / unrecognised	0	0	sees nothing

Cross-check: signed in as Oxbridge, the system returned Oxbridge's 3 students and **zero** of Metro's — confirming no leakage between institutions in the shared database. The full automated test suite (558 checks) passes, including a dedicated isolation test.

What this guarantees, and how it was delivered

Security guarantees

- One institution can never read another's data — enforced by the database.
- One institution can never create or change another's data (write-side check).
- A login for one institution cannot be replayed on another's web address.
- A production-grade "fail-closed" mode is available: if the institution is ever unknown, the answer is *nothing*, never *everything*.

Delivered in stages

Foundation	tag core records by institution
Enforcement	database-level isolation switched on
Coverage	rolled out to all 95 data tables
Addressing	per-institution subdomains + guard
Hardening	fail-closed application role

In plain terms

The platform has moved from a single-institution system to a **true multi-tenant platform**: many institutions, one system, with a hard and independently-proven boundary between them — ready to onboard new customers as a configuration step rather than a build.

Appendix A — Entity-relationship model (core)

Generated directly from the live database schema: real tables, columns and types, with primary keys (**PK**) and foreign keys (**FK**) marked. Teal links are the `tenant_id → tenant` relationship every institution-owned table carries; grey links are the domain relationships around the `student` record.

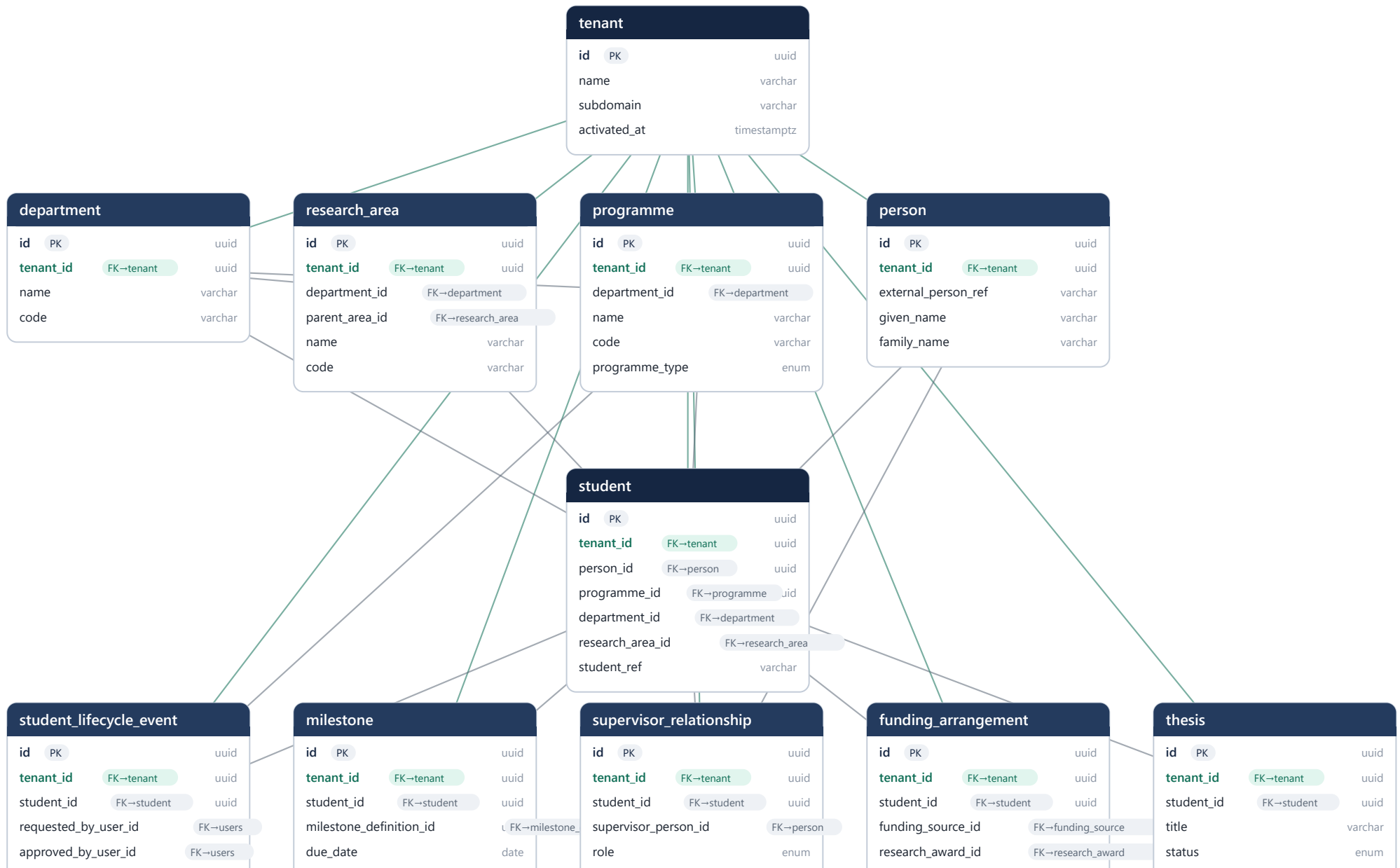


Figure 5 — L1: Core entity-relationship model (11 representative tables of 107). Every table references **tenant**.

Appendix A2 — Full entity-relationship model (all tables)

The complete map for documentation: all 106 data tables and all 261 foreign keys, laid out as a reference. It is intentionally dense — not meant to be read line-by-line, but to show the full shape of the schema. Tenant-scoped tables are outlined in teal; the nine shared/global tables (roles, permissions, sign-in tokens, statutory specs, the tenant directory) are filled navy. Faint teal lines are the universal `tenant_id` → `tenant` relationship; grey lines are domain foreign keys.

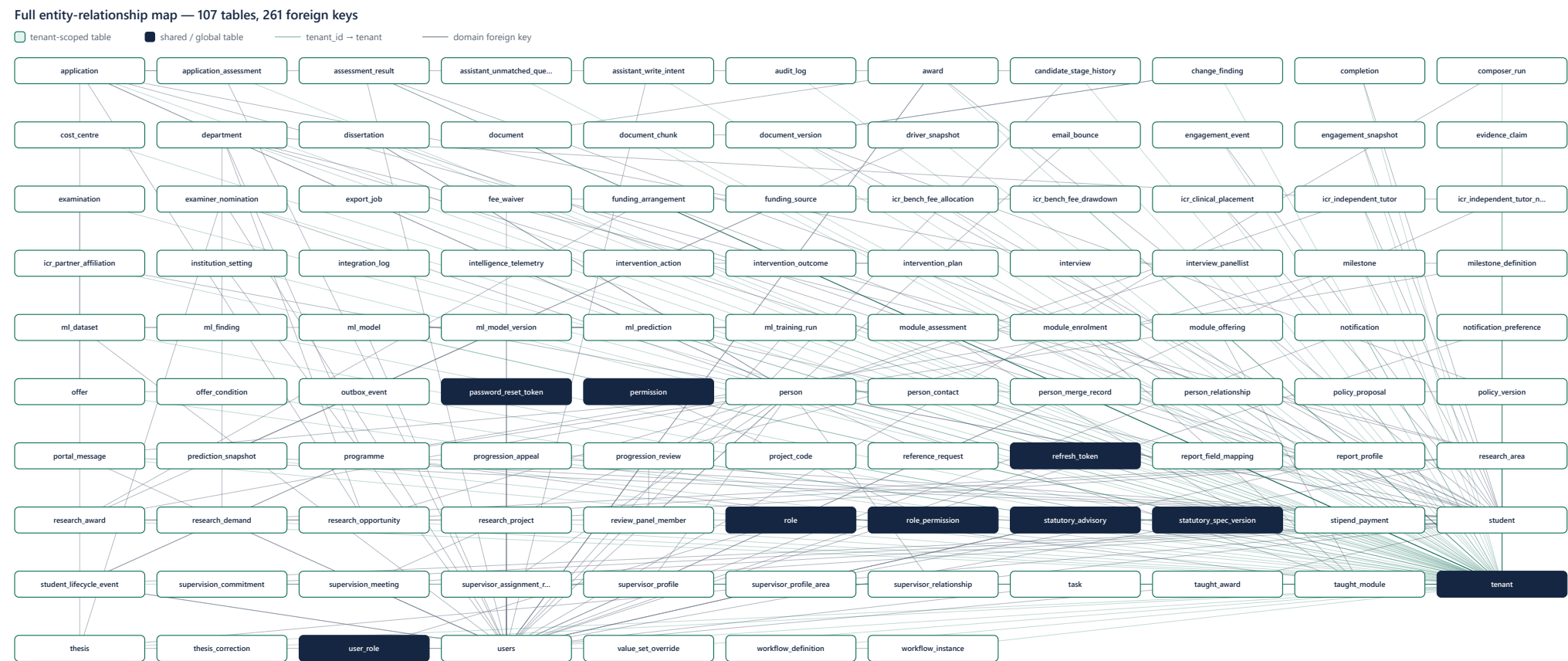


Figure 6 — L2: Full schema map — every table and every foreign key.

Appendix B — All tables and foreign keys

The complete schema: every one of the 107 tables, its column count, whether it is tenant-scoped (carries `tenant_id`), and its real foreign-key relationships (261 in total). This is the authoritative list behind the diagrams above.

Table	Cols	Tenant	Foreign keys (column → referenced table)
alembic_version	1	–	
application	14	✓	person_id→person, research_area_id→research_area, research_opportunity_id→research_opportunity, tenant_id→tenant
application_assessment	10	✓	application_id→application, assessor_user_id→users, tenant_id→tenant
assessment_result	14	✓	assessment_id→module_assessment, marked_by_user_id→users, module_enrolment_id→module_enrolment, tenant_id→tenant
assistant_unmatched_query	11	✓	tenant_id→tenant
assistant_write_intent	12	✓	proposed_by_user_id→users, tenant_id→tenant
audit_log	12	✓	tenant_id→tenant
award	14	✓	certificate_document_id→document, confirmed_by_user_id→users, proposed_by_user_id→users, student_id→student, tenant_id→tenant
candidate_stage_history	10	✓	application_id→application, moved_by_user_id→users, tenant_id→tenant
change_finding	15	✓	tenant_id→tenant, version_a_id→document_version, version_b_id→document_version
completion	9	✓	student_id→student, tenant_id→tenant
composer_run	16	✓	tenant_id→tenant, user_id→users
cost_centre	6	✓	tenant_id→tenant
department	6	✓	tenant_id→tenant
dissertation	15	✓	second_marker_person_id→person, student_id→student, supervisor_person_id→person, tenant_id→tenant
document	14	✓	tenant_id→tenant
document_chunk	11	✓	tenant_id→tenant, version_id→document_version
document_version	14	✓	student_id→student, tenant_id→tenant
driver_snapshot	11	✓	prediction_id→prediction_snapshot, tenant_id→tenant
email_bounce	9	✓	person_id→person, tenant_id→tenant
engagement_event	10	✓	student_id→student, tenant_id→tenant
engagement_snapshot	10	✓	student_id→student, tenant_id→tenant
evidence_claim	14	✓	tenant_id→tenant
examination	11	✓	tenant_id→tenant, thesis_id→thesis
examiner_nomination	12	✓	approved_by_user_id→users, examiner_person_id→person, tenant_id→tenant, thesis_id→thesis
export_job	10	✓	tenant_id→tenant
fee_waiver	14	✓	approved_by_user_id→users, arrangement_id→funding_arrangement, student_id→student, tenant_id→tenant
funding_arrangement	18	✓	funding_source_id→funding_source, research_award_id→research_award, student_id→student, tenant_id→tenant
funding_source	6	✓	tenant_id→tenant
icr_bench_fee_allocation	12	✓	funding_source_id→funding_source, student_id→student, tenant_id→tenant
icr_bench_fee_drawdown	10	✓	allocation_id→icr_bench_fee_allocation, tenant_id→tenant

Table	Cols	Tenant	Foreign keys (column → referenced table)
icr_clinical_placement	13	✓	student_id→student, tenant_id→tenant
icr_independent_tutor	9	✓	student_id→student, tenant_id→tenant, tutor_department_id→department, tutor_person_id→person
icr_independent_tutor_note	8	✓	authored_by_user_id→users, tenant_id→tenant, tutor_id→icr_independent_tutor
icr_partner_affiliation	12	✓	student_id→student, tenant_id→tenant
institution_setting	7	✓	tenant_id→tenant, updated_by_user_id→users
integration_log	11	✓	tenant_id→tenant
intelligence_telemetry	16	✓	tenant_id→tenant
intervention_action	15	✓	plan_id→intervention_plan, tenant_id→tenant
intervention_outcome	9	✓	plan_id→intervention_plan, tenant_id→tenant
intervention_plan	14	✓	student_id→student, tenant_id→tenant
interview	10	✓	application_id→application, tenant_id→tenant
interview_panellist	7	✓	interview_id→interview, person_id→person, tenant_id→tenant
milestone	10	✓	milestone_definition_id→milestone_definition, student_id→student, tenant_id→tenant
milestone_definition	13	✓	programme_id→programme, tenant_id→tenant
ml_dataset	15	✓	created_by_user_id→users, tenant_id→tenant
ml_finding	12	✓	dataset_id→ml_dataset, tenant_id→tenant
ml_model	8	✓	created_by_user_id→users, tenant_id→tenant
ml_model_version	19	✓	decided_by_user_id→users, model_id→ml_model, tenant_id→tenant, training_run_id→ml_training_run
ml_prediction	12	✓	model_id→ml_model, model_version_id→ml_model_version, student_id→student, tenant_id→tenant
ml_training_run	11	✓	dataset_id→ml_dataset, model_id→ml_model, started_by_user_id→users, tenant_id→tenant
module_assessment	13	✓	module_id→taught_module, tenant_id→tenant
module_enrolment	12	✓	module_id→taught_module, student_id→student, tenant_id→tenant
module_offering	6	✓	module_id→taught_module, programme_id→programme, tenant_id→tenant
notification	9	✓	recipient_user_id→users, tenant_id→tenant
notification_preference	10	✓	tenant_id→tenant, user_id→users
offer	10	✓	application_id→application, tenant_id→tenant
offer_condition	10	✓	offer_id→offer, tenant_id→tenant
outbox_event	12	✓	tenant_id→tenant
password_reset_token	6		user_id→users
permission	5		–
person	12	✓	tenant_id→tenant
person_contact	10	✓	person_id→person, tenant_id→tenant
person_merge_record	9	✓	merged_by_user_id→users, tenant_id→tenant
person_relationship	9	✓	person_id→person, tenant_id→tenant
policy_proposal	12	✓	source_version_id→policy_version, tenant_id→tenant
policy_version	8	✓	document_version_id→document_version, tenant_id→tenant
portal_message	10	✓	author_user_id→users, student_id→student, supervisor_person_id→person, tenant_id→tenant

Table	Cols	Tenant	Foreign keys (column → referenced table)
prediction_snapshot	11	✓	model_version_id→ml_model_version, student_id→student, tenant_id→tenant
programme	12	✓	department_id→department, tenant_id→tenant
progression_appeal	12	✓	decided_by_user_id→users, review_id→progression_review, student_id→student, tenant_id→tenant
progression_review	16	✓	decided_by_user_id→users, milestone_id→milestone, tenant_id→tenant
project_code	6	✓	tenant_id→tenant
reference_request	14	✓	application_id→application, tenant_id→tenant
refresh_token	6		user_id→users
report_field_mapping	12	✓	profile_id→report_profile, tenant_id→tenant
report_profile	14	✓	signed_off_by→users, tenant_id→tenant
research_area	8	✓	department_id→department, parent_area_id→research_area, tenant_id→tenant
research_award	16	✓	funder_id→funding_source, principal_investigator_id→person, tenant_id→tenant
research_demand	13	✓	department_id→department, raised_by_user_id→users, research_area_id→research_area, research_award_id→research_award, tenant_id→tenant
research_opportunity	19	✓	department_id→department, principal_supervisor_id→person, research_area_id→research_area, research_award_id→research_award, research_demand_id→research_demand, tenant_id→tenant
research_project	12	✓	research_area_id→research_area, research_award_id→research_award, research_opportunity_id→research_opportunity, student_id→student, tenant_id→tenant
review_panel_member	8	✓	person_id→person, review_id→progression_review, tenant_id→tenant
role	5		–
role_permission	2		permission_id→permission, role_id→role
statutory_advisory	18		created_by→users, decided_by→users
statutory_spec_version	13		accepted_by→users, source_advisory_id→statutory_advisory
stipend_payment	14	✓	arrangement_id→funding_arrangement, student_id→student, tenant_id→tenant
student	15	✓	department_id→department, person_id→person, programme_id→programme, research_area_id→research_area, tenant_id→tenant
student_lifecycle_event	25	✓	approved_by_user_id→users, new_programme_id→programme, previous_programme_id→programme, requested_by_user_id→users, student_id→student, tenant_id→tenant
supervision_commitment	15	✓	meeting_id→supervision_meeting, student_id→student, tenant_id→tenant
supervision_meeting	15	✓	recorded_by_user_id→users, student_id→student, supervisor_person_id→person, tenant_id→tenant
supervisor_assignment_request	17	✓	decided_by_user_id→users, proposed_supervisor_person_id→person, requested_by_user_id→users, reviewed_by_user_id→users, student_id→student, tenant_id→tenant
supervisor_profile	11	✓	person_id→person, tenant_id→tenant
supervisor_profile_area	6	✓	research_area_id→research_area, supervisor_profile_id→supervisor_profile, tenant_id→tenant
supervisor_relationship	12	✓	student_id→student, supervisor_person_id→person, tenant_id→tenant
task	16	✓	assignee_user_id→users, tenant_id→tenant
taught_award	10	✓	decided_by_user_id→users, student_id→student, tenant_id→tenant
taught_module	12	✓	convenor_person_id→person, programme_id→programme, tenant_id→tenant
tenant	8		–

Table	Cols	Tenant	Foreign keys (column → referenced table)
thesis	10	✓	student_id→student, tenant_id→tenant
thesis_correction	10	✓	approved_by_user_id→users, tenant_id→tenant, thesis_id→thesis
user_role	2		role_id→role, user_id→users
users	11	✓	person_id→person, tenant_id→tenant
value_set_override	10	✓	tenant_id→tenant, updated_by_user_id→users
workflow_definition	11	✓	tenant_id→tenant
workflow_instance	9	✓	definition_id→workflow_definition, tenant_id→tenant